

Distributed Systems



1. System Foundations & Architecture

Brief: A distributed system (DS) acts as a single computer to the user, but is actually multiple separate machines connected via a network. The goal is to share resources without the user knowing they are distributed.

A. Computing Types Tree

```
├─ Computing Architectures
│   ├── Centralized Computing
│   │   └─ All processing/storage happens on one single mainframe. (High risk of failure)
│   ├── Decentralized Computing
│   │   └─ Multiple localized hubs. (Better, but hubs can still fail)
│   └─ Distributed Computing
│       └─ Fully interconnected nodes working concurrently. (Highly scalable, fault-tolerant)
```

B. The 5 Core Characteristics

Feature	What it means	How it's achieved	Example
Resource Sharing	Hardware, data, and software are shared.	IPC (Inter-process Communication) & Network routing.	Cloud Storage (AWS), Shared Printers.
Concurrency	Multiple tasks running at the exact same time.	Multithreading & multi-core processors.	Two users editing a Google Doc simultaneously.
Scalability	Handling growth without slowing down.	Adding more independent nodes to the network.	Scaling from 10 users to 1,000,000 users seamlessly.
Fault Tolerance	System survives if a computer/link dies.	Replication (copying data) & Redundancy.	If Server A crashes, Server B immediately takes over.
Transparency	Hiding the complex network from the user.	Virtual namespaces and unified interfaces.	You don't know <i>which</i> Google server in the world holds your email.

C. Architectural Models (Client/Server vs P2P)

Model	Brief Concept	Roles & Rules	Tiers / Variations
Client-Server	Centralized control over distributed resources.	Static: Servers host the data/logic. Clients request it.	2-Tier: UI ↔ Database. 3-Tier: UI ↔ App Logic ↔ Database.

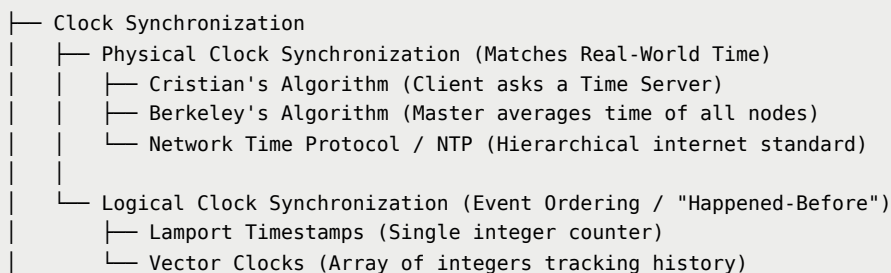
Peer-to-Peer (P2P)	Fully decentralized network. Equal responsibility.	Dynamic: Every node is a "Servent" (Acts as both (SERVer + cliENT)).	Pure: No central server at all (BitTorrent). Hybrid: Central index server tracks users, but file transfer is P2P.
---------------------------	--	---	--



2. Time, Clocks & Synchronization

Brief: Because every computer has its own physical quartz crystal, their times drift apart due to temperature and hardware differences. We must synchronize them either to real-world time (Physical) or simply agree on the order of events (Logical).

A. The Clock Synchronization Tree



B. Physical Clocks (Real-World Time)

Algorithm	How it Works	The Math / Key Concept	Best For
Cristian's	Client asks Time Server. Compensates for the network travel time.	$New_Time = Server_Time + \frac{RTT}{2}$	Simple LANs with a single accurate time server.
Berkeley	A Master node polls all Slaves, calculates the average time, and tells slaves to speed up/slow down.	$Offset = Average - Node_Time$ (Master pushes offsets, not actual time).	Closed networks without a perfect external UTC source.
NTP	Global tree of servers. Stratum 0 (Atomic), Stratum 1 (Primary), Stratum 2+ (Secondary).	Delay: $(T_4 - T_1) - (T_3 - T_2)$ Offset: $\frac{(T_2 - T_1) + (T_3 - T_4)}{2}$	The entire Internet.

C. Logical Clocks (Event Ordering)

Algorithm	Mechanism	The Rule	Pros & Cons
-----------	-----------	----------	-------------

Lamport	Passes a single integer.	1. Increment before event. 2. On receive: $Clock = \max(Local, Message) + 1$	Pro: Extremely lightweight. Con: If $A < B$, it <i>doesn't</i> mean A caused B. Cannot prove causality.
Vector	Passes an array of integers [Node1, Node2] .	On receive, take the MAX of every individual index, then add 1 to your own index.	Pro: 100% proves exact causal history. Con: Array size grows massive with N nodes (heavy overhead).



3. Operating Systems & Processes

Brief: The OS manages how the local machine handles the distributed workload. Modern distributed systems rely on lightweight threads and minimal kernels to maximize network speed.

A. OS Architecture Tree

```

— Operating System Architecture
|   — Monolithic Kernel (Linux/Windows)
|   |   — Massive. Does everything (memory, file, network). Bad for distributed modularity.
|   — Microkernel (Distributed OS Standard)
|   |   — Tiny core (IPC & Scheduling only). Everything else runs safely in user-space.
|   — Container OS (Modern Cloud)
|   |   — Immutable. Runs apps in isolated namespaces (Docker/Kubernetes).
```

IPC Mechanisms (How processes talk):

- **Pipes:** Unidirectional flow. Used for parent-child processes on the *same* machine.
- **Shared Memory:** Processes map a common block of RAM. Fastest, but requires strict mutual exclusion.
- **Sockets:** Bidirectional endpoint communication. Used for processes on *different* machines over a network (TCP/UDP).

B. Processes vs. Threads

Feature	Process	Thread
Definition	An independent program in execution.	A lightweight path of execution inside a process.
Memory	Highly isolated. Own memory space.	Shared. Shares memory with sibling threads.

Context Switching	Slow and resource-heavy (OS must flush cache).	Fast (No memory flush required).
Failure Impact	If it crashes, others are completely safe.	If one thread crashes, the whole process can die.
Distributed Use	Securely isolating user containers (Docker).	Handling thousands of concurrent server requests (Web Servers).



4. Communication & Network Layers

Brief: Nodes must communicate across physical wires. The network stack wraps raw data in layers of routing information. To the application layer, it should look like the nodes are talking directly.

A. The Network Stack Tree

```

├─ Network Protocol Models
│   ├── OSI Model (7 Layers - Theoretical Framework)
│   │   ├── 7. Application, 6. Presentation, 5. Session (Data)
│   │   ├── 4. Transport (Segments)
│   │   ├── 3. Network (Packets)
│   │   └── 2. Data Link(Frames), 1. Physical (Bits)
│   └── TCP/IP Model (4 Layers - Actual Internet Implementation)
│       ├── Application (HTTP, DNS, gRPC)
│       ├── Transport (TCP, UDP)
│       ├── Internet (IPv4, IPv6)
│       └── Network Access (MAC Addresses, Ethernet, Fiber)

```

B. Transport Protocols Comparison

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection	Connection-Oriented (3-way handshake).	Connectionless (Fire-and-forget).
Reliability	Highly Reliable (Tracks packets, retransmits lost ones).	Unreliable (Packets can drop or arrive out of order).
Speed/Overhead	Slower, heavy header overhead.	Extremely fast, minimal overhead.
DS Use Case	File transfers, RPC, Web browsing (HTTP/2).	WebRTC, DNS lookups, Live video streaming.

C. Remote Procedure Call (RPC)

Brief: Allows a program to execute a function on a remote machine as if it were a local function. It hides the complexity of network communication.

Component	Function in the RPC Flow
-----------	--------------------------

Client Stub	Acts as a proxy. Takes the local function call and Marshals (serializes) the parameters into a network message.
RPC Runtime	Handles the actual OS-level transmission of the message over the network.
Server Stub	Receives the message, Unmarshals it, and calls the actual server function.
Execution	Server executes the logic and returns the result back through the reverse path.



5. Mutual Exclusion (Ricart-Agrawala)

Brief: When multiple nodes want to access the same shared resource (a Critical Section) at the exact same time, they need an algorithm to prevent collisions without relying on a central traffic-cop server.

The Fully Distributed Execution Flow

- The Request:** Node A wants the resource. It broadcasts a **REQUEST** message containing its Node ID and its current **Lamport Timestamp** to *every* other node.
- The Reply Logic:** When Node B receives Node A's request, it checks its own state:
 - State 1:* If Node B does NOT want the resource → Replies **OK**.
 - State 2:* If Node B is CURRENTLY USING the resource → Does nothing (queues Node A).
 - State 3:* If Node B ALSO WANTS the resource → Compares timestamps. The older (smaller) timestamp wins. If Node A is older, Node B replies **OK**. If Node B is older, it queues Node A.
- Execution:** Node A can only enter the Critical Section once it has received an **OK** from $N - 1$ nodes (everyone else).
- Release:** Once Node A finishes, it sends an **OK** to all the nodes it placed in its queue.
- Message Complexity:** Requires exactly $2(N - 1)$ messages per critical section entry ($N - 1$ Requests broadcasted + $N - 1$ Replies received).



6. Distributed File Systems (DFS) & Security

Brief: A DFS stores files across multiple machines but makes it look like a single local hard drive. It requires strict permission architectures to ensure clients only access what they are authorized to see.

A. Directory Structures

Structure	Design Concept	Pros	Cons
-----------	----------------	------	------

Hierarchical	Tree. A file/folder has exactly <i>one</i> parent directory.	Simple logic, easy to traverse and manage.	Makes it very difficult for multiple users to share a single file.
Acyclic Graph	Network. A file can have <i>multiple</i> parent directories (shortcuts/symlinks).	Excellent for collaborative environments and shared files.	Complex deletion. If Parent A deletes the file, Parent B is left with a broken pointer.

B. Access Control Models

Model	Where is the rule stored?	Analogy
Access Control Lists (ACLs)	Attached to the File .	A bouncer checking a guest list at the door of a club.
Capabilities	Attached to the User/Process .	A user holding a cryptographically signed movie ticket.

C. The Ultimate File Permission Table (Octal vs Symbolic)

Brief: Permissions are calculated using the 4-2-1 rule. **Read = 4** , **Write = 2** , **Execute = 1** . These are applied in three distinct blocks: **Owner**, **Group**, and **Others** (e.g., **755**).

Octal Num	Calculation	Symbolic	What it actually allows a user to do
0	0	---	No Access: The user cannot interact with the file at all.
1	1	--X	Execute Only: Can run a compiled program, but cannot read its code. (Secure binaries).
2	2	-W-	Write Only: Can append data to a file blindly without reading it. (Drop-boxes/Logs).
3	2 + 1	-wX	Write + Execute: Can run the file and write to it, but cannot read the contents.
4	4	r--	Read Only: Can open and view the file, but cannot alter or run it. (Standard documents).
5	4 + 1	r-X	Read + Execute: Can read the code and run it. (Standard requirement for Scripts).
6	4 + 2	rw-	Read + Write: Can view and edit the file, but cannot run it as a program. (Standard editable text file).
7	4 + 2 + 1	rwX	Full Control: Unrestricted access to read, modify, and execute.

Common Permission Configurations:

- 700 (rwx-----):** Only the Owner has access. Complete privacy.

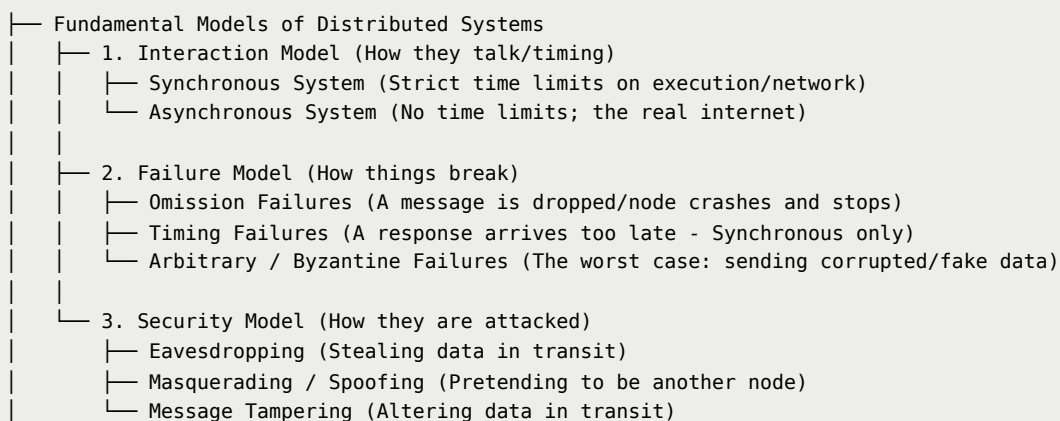
- **755** (`rwxr-xr-x`): Owner can do everything. Everyone else can only read and run it. (Standard script deployment).
- **644** (`rw-r--r--`): Owner can edit. Everyone else can only read. (Standard web server files like HTML/CSS).
- **777** (`rw-rwxrwx`): Total anarchy. Anyone on the network can read, edit, or delete the file.



7. Fundamental System Models

Brief: While *Architectural* models (C/S vs P2P) define how nodes are physically arranged, *Fundamental* models define how the system actually behaves under stress—specifically how nodes interact, how they fail, and how they defend against attacks.

A. The Fundamental Models Tree



B. Synchronous vs. Asynchronous Systems

Feature	Synchronous Distributed System	Asynchronous Distributed System
Time Limits	Known, strict upper/lower bounds on processing and network delays.	No bounds. A message could take 1ms or 1 hour.
Clock Drift	Clock drift rates are known and bounded.	Clock drift is arbitrary.
Real-World Use	Hard real-time systems (Aircraft controls, pacemakers).	The Internet. Most distributed systems operate here.
Failure Detection	Easy. If a node doesn't reply in X seconds, it is dead.	Very hard. Is the node dead, or is the network just slow?

Replication Models (Handling Failures):

- **Active Replication:** Every client request is sent to *all* replica servers simultaneously. They all process it. If one dies, the others already have the result. (High bandwidth,

zero downtime).

- **Passive (Primary-Backup) Replication:** Client only talks to the Primary server. The Primary processes it, updates its own state, and then secretly forwards the update to the Backups. (Lower bandwidth, slight delay if the Primary dies).



8. Coordination: Leader Election

Brief: Many distributed algorithms (like Mutual Exclusion or Berkeley's Clock) require a single "Coordinator" or "Master" node. If the Master crashes, the remaining nodes must run an Election Algorithm to choose a new Master fairly. **Rule: The node with the highest ID always wins.**

A. The Bully Algorithm

- **Design:** Aggressive and direct. Nodes bully lower-ID nodes into submission.
- **The Flow:**
 1. Node 4 notices the Master is dead. It sends an **ELECTION** message to all nodes with an ID *higher* than 4 (Nodes 5, 6, 7).
 2. If Node 5 is alive, it replies **OK** (shutting Node 4 up) and starts its own election by messaging 6 and 7.
 3. This continues until the absolute highest alive node (e.g., Node 7) receives no **OK** replies because no one is higher than it.
 4. Node 7 broadcasts a **COORDINATOR** message to everyone, declaring victory.
 5. **Worst-Case Complexity:** $O(N^2)$ messages if the lowest ID node starts the election.

B. The Ring Algorithm

- **Design:** Peaceful and structured. Nodes are arranged in a logical circle.
- **The Flow:**
 1. Node 4 notices the Master is dead. It starts an **ELECTION** message containing its own ID **[4]** and passes it to its neighbor.
 2. As the message travels around the ring, every active node appends its own ID to the list (e.g., **[4, 5, 2, 7]**).
 3. When the list makes a full circle back to Node 4, it looks at the array, picks the highest number **[7]**, and changes the message to **COORDINATOR: 7**.
 4. It sends this message around the ring one more time to inform everyone.

5. **Worst-Case Complexity:** $2N$ messages to complete the cycle and announce the new coordinator.

9. Distributed Shared Memory (DSM) & Consistency

Brief: DSM gives distributed computers the illusion that they share a single pool of RAM. Because they are actually caching data locally, the system needs strict "Consistency Models" to resolve what happens when two nodes edit the same variable.

Sequential vs. Release Consistency (PYQ Favorite)

Consistency Model	The Rule	Analogy	Performance
Sequential Consistency	Every node must see exactly the same sequence of memory operations in the exact same order.	A single-file checkout line. Everyone agrees on who went first, second, and third.	Slow. Heavy network overhead to keep every node perfectly synced at all times.
Release Consistency	Nodes can do whatever they want locally. Memory is only synced when a node explicitly uses an Acquire or Release lock.	Working on a local git branch, and only syncing with the team when you explicitly git push .	Very Fast. Minimal network traffic; syncs only happen at critical moments.

10. Stream Synchronization (Multimedia)

Brief: Unlike text/files (which use discrete TCP), continuous media (audio/video) uses streams that must be perfectly synchronized to prevent jitter and lip-sync failure.

The Mathematics of Lip-Sync

To achieve lip-sync, the OS must mathematically match the sampling rate of the audio to the frame rate of the video.

- **Audio Quality:** CD-quality audio is sampled at 44.1 kHz (44,100 samples per second) using 16-bit samples.
 - *Sync requirement:* Must process an audio data unit every 22.6 microseconds.
- **Video Quality:** NTSC standard video runs at 30 Hz (30 frames per second).
 - *Sync requirement:* Must process a video frame every 33.33 milliseconds.

- **The Synchronization Formula:** The middleware forces synchronization by tying the two streams together. It mathematically matches **1 video frame** to exactly 1,470 **audio samples** to keep lip movement and sound perfectly aligned.

11. Distributed Transactions & Consensus

Brief: When a transaction spans multiple machines (e.g., transferring money from Server A to Server B), the system must guarantee the **ACID** properties (Atomicity, Consistency, Isolation, Durability). Either *all* servers commit the change, or *none* of them do.

The Two-Phase Commit Protocol (2PC)

The Flow:

- **Phase 1: The Voting Phase**
 1. The Coordinator sends a **PREPARE** request to all participating nodes.
 2. Each node checks if it can safely execute the transaction.
 3. Nodes reply with **VOTE_COMMIT** (Yes) or **VOTE_ABORT** (No).
- **Phase 2: The Completion Phase**
 1. If the Coordinator receives **VOTE_COMMIT** from **ALL** nodes, it broadcasts a **GLOBAL_COMMIT** message.
 2. If even **ONE** node replies **VOTE_ABORT** (or times out), the Coordinator broadcasts a **GLOBAL_ABORT** to roll everything back.

The Flaw (Blocking Problem): If the Coordinator crashes exactly after everyone votes **VOTE_COMMIT** but *before* it sends the **GLOBAL_COMMIT**, all nodes are locked indefinitely waiting for the final verdict. (This is solved by the heavier Three-Phase Commit, 3PC).

12. Distributed Deadlocks

Brief: In a single OS, a deadlock happens when Process A holds Resource 1 and waits for Resource 2, while Process B holds Resource 2 and waits for Resource 1. In a Distributed System, this cycle spans across the network, making it invisible to any single machine.

Deadlock Detection Strategies

Strategy	How it works	Pros & Cons
Centralized Detection	All nodes send their local dependency graphs to one central server. The server stitches	Pro: Simple to implement. Con: The central server

	them into a Global Wait-For Graph (WFG) and checks for cycles.	becomes a bottleneck and a single point of failure.
Edge-Chasing (Distributed)	When a node suspects a deadlock, it sends a special PROBE message containing its ID along the dependency chain.	Pro: No central server needed. Con: High network traffic.

The Edge-Chasing Rule: If Node A sends a **PROBE(A)** to Node B, and Node B passes it to Node C, and eventually Node A receives its *own* **PROBE(A)** message back, a circular deadlock is confirmed.



13. The CAP Theorem (Brewer's Theorem)

Brief: The holy grail rule of modern distributed system design. It states that a distributed data store cannot simultaneously guarantee more than two out of the following three properties.

The 3 Properties

Property	Definition	What it means in practice
Consistency (C)	Every read receives the most recent write, or an error.	If I update my profile on Node A, a user reading from Node B instantly sees the update.
Availability (A)	Every request receives a non-error response (but without the guarantee that it contains the most recent write).	The system never goes down. It will always give you data, even if it is slightly outdated.
Partition Tolerance (P)	The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.	If the network cable between Data Center 1 and Data Center 2 is cut, the system survives.

The Reality of CAP:

Because network failures (Partitions) are inevitable in the real world, a distributed system **must** choose **P**. Therefore, architects must trade off between **C** and **A**:

- **CP Systems (Consistency + Partition Tolerance):** If the network breaks, the system shuts down nodes to prevent outdated data from being read. (Good for banking/finance).
- **AP Systems (Availability + Partition Tolerance):** If the network breaks, nodes stay online and serve whatever data they have, even if it is stale. (Good for social media feeds/e-commerce).